Programming and Data Structures - II (CS3201)

Lecture 5

Kripabandhu Ghosh

CDS, IISER Kolkata

HASHING

Motivation

Conventional Search

- The search time of algorithms like linear search (time complexity $\mathcal{O}(n)$), binary search (if array is sorted on the key; time complexity $\mathcal{O}(logn)$)) etc. is dependent on input size (n)
- Quest for a search algorithm with time complexity $\mathcal{O}(1)$

Possible solution

Direct-address Table

Direct-address Table

Example Scheme

- A database of n employee names is stored in a table T (implemented by an array)
- For each employee id (i) the employee name is given by $T[i]$

Example ($n = 5$)

Empid	**Employee Name**
0	Albert Einstein
1	Isaac Newton
2	Srinavasa Ramanujan
3	Richard Feynman
4	Linus Pauling

Direct-address Table (formal definition)

Premise

- **U** = {1, 2,..., *m* - 1} is the universe of keys (all possible values), *m* is small
- **K** ⊆ **U**, **K** is the set of keys actually stored
- **T[0...** *m* **- 1]**: direct-access table such that *k* ∈ **K** is stored in the *slot* for *k* (**T[k]**)

Search operation

DIRECT-ACCESS-SEARCH(T, k)
  return T[k]

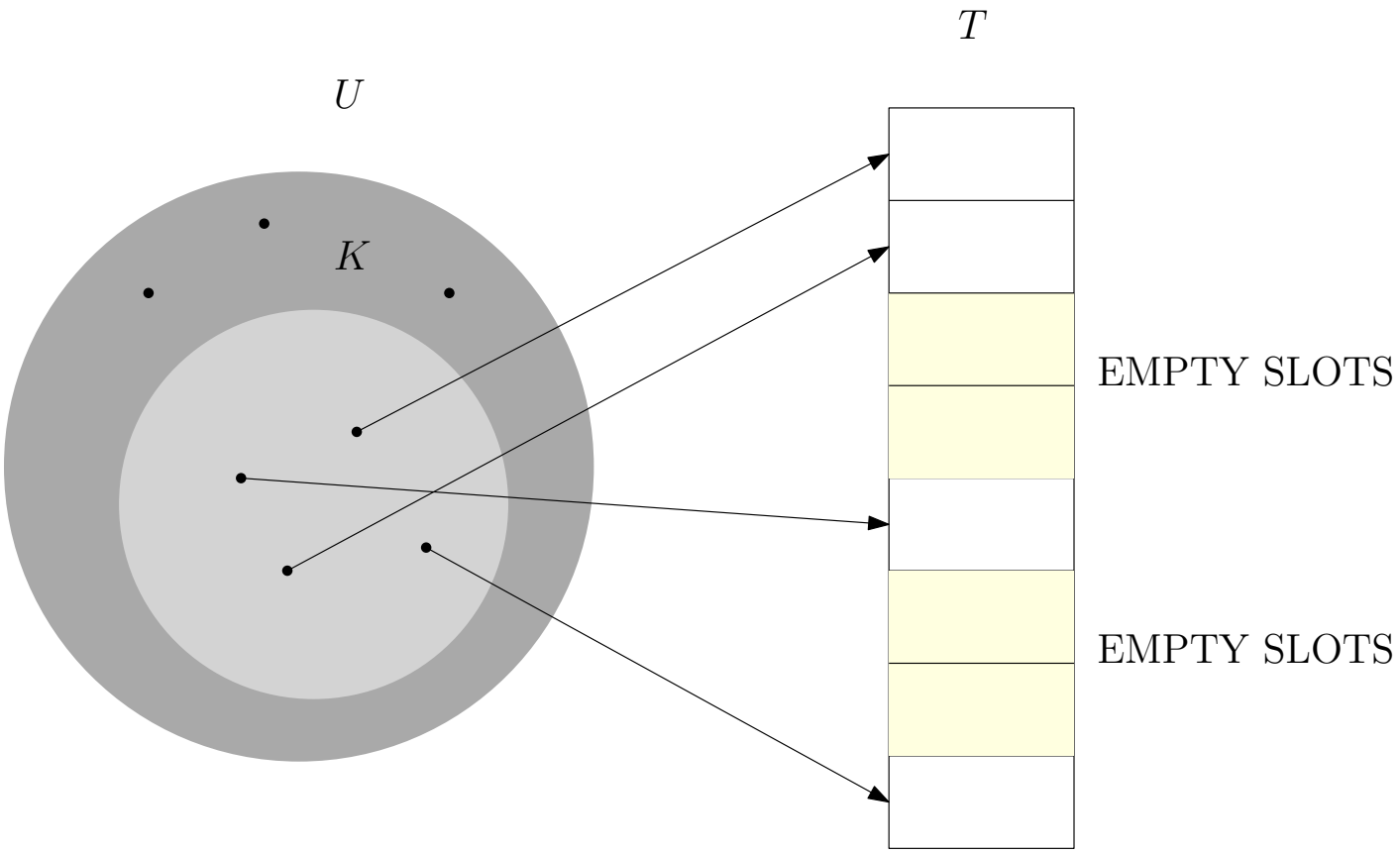
Insert operation

DIRECT-ACCESS-INSERT(T, x)
  T[x.key] = x

Delete operation

DIRECT-ACCESS-DELETE(T, x)
  T[x.key] = NIL

Direct-address Table (visualization)



Direct-address Table: characteristics

Advantages

- Ease of access
- Search, Insert and Delete operations take $\mathcal{O}(1)$ time

Disadvantages

- If $|U|$ is large, maintaining T is expensive
- If $|K| \ll |U|$, much space is wasted
  - If the company has 1000 employees with ids in the range $0 - 1000000$, T will unnecessarily have around 10^6 entries out of which only 1000 will be used!!

Hash Table (formal definition)

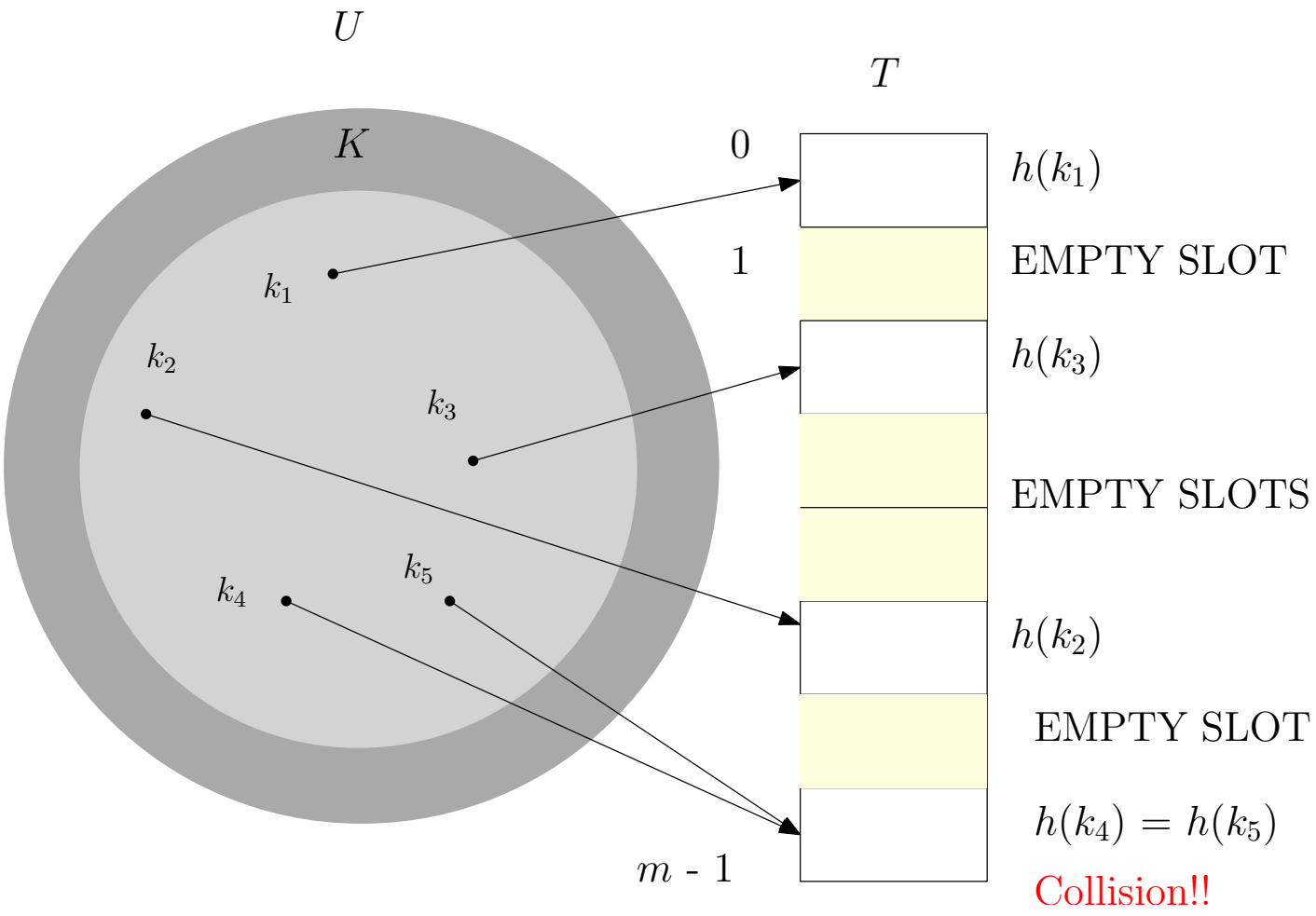
Premise

- **U** $= \{1, 2, \ldots, m - 1\}$ is the universe of keys (all possible values), m is small
- **K** $\subseteq$ **U**, **K** is the set of keys actually stored
- **T[0... m - 1]**: hash table such that $k \in$ **K** is stored in the *slot* for k given by **h[k]** where h is the **hash function** defined as:

$$h\colon U \rightarrow \{0, 1, \ldots, m - 1\}$$

- $m \ll |U|$
- $h[k]$ is called the **hash value** of the key k

Hash Table (visualization)



Hash Table : collision

Condition

For k_i, $k_j \in U$ $(k_i \neq k_j)$, $h(k_i) = h(k_j)$

Avoidance

- h an be chosen as random or uniform (any element in U is equally likely to hash into the m slots)
- However, since m $< |U|$, collisions are practically impossible to avoid

Resolution

- Chaining
- Open addressing

Hash function

Desired properties

- Fast (and easy) to compute
- Minimize collision (enable uniform distribution of the keys in the hash table)

Examples

- Truncation (extraction)
- Midsquare method
- Division method
- Multiplication method

Hash function: Truncation (extraction)

Scheme

Take a part of a (numeric) key

Example

- **Keys**: 8775664, 123456, 566789
- **Hash table**: 100 slots
- **Hash function (h)**: Choose the last two digits
- **Hash addresses**: $h(8775664) = 64$, $h(123456) = 56$, $h(566789) = 89$

Features

- **Advantage**: Easy to calculate
- **Disadvantage**: Prone to collisions (e.g. keys 8775664, 123464, 566764 will have the same address, i.e. 64)

Hash function: Mid-square method

Invented by John von Neumann in 1949 for pseudo-random number sequence generation

Scheme

1. Square (numeric) key
2. Take the middle k digits of the square

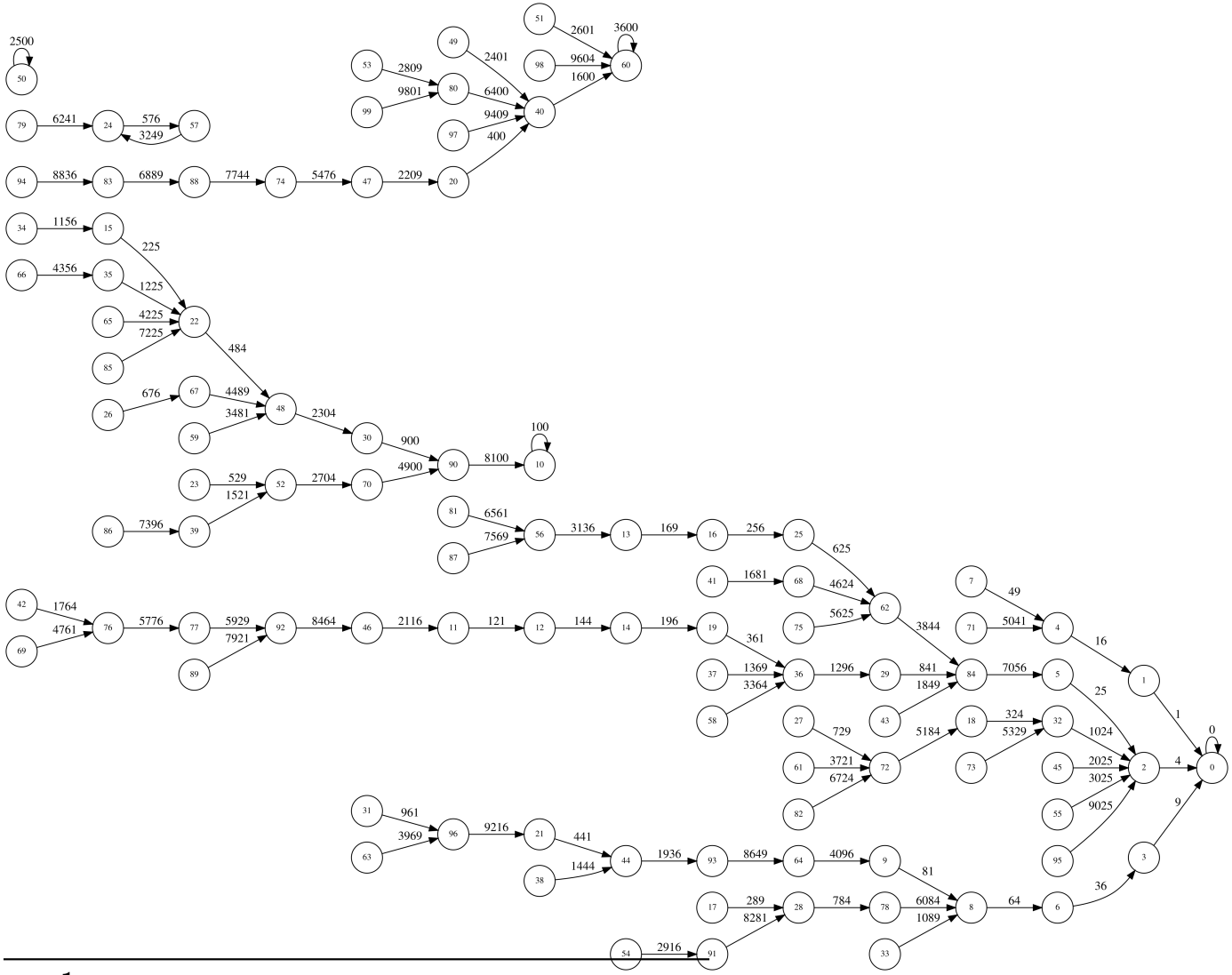
Example

- For the key 1273, 1273*1273 = 16**205**29 $\Rightarrow$ 205 (address)
- If h is the hash function, $h(1273) = 205$

Features

- If the key is too large, part of the key can be chosen and squared
- **Advantage**: Simple
- **Disadvantage**: Starts repeating after short iterations

Hash function: Mid-square method (cycles for key of length 2)[1]



[1] https://hbfs.wordpress.com/2017/11/21/the-middle-square-method-generating-random-sequences-viii/

Hash function: Modulo division

Scheme

$$h(k) = k \bmod m$$

- k is the key
- m is the number of slots

Example

For the key $k = 100$, $m = 12$, $h(k) = 4$

Features

- **Advantage**: Simple and fast
- **Disadvantage**: Certain m (e.g. 2^p) values should be avoided
- A prime number not too close to the exact power of 2 often is a good choice for m

Hash function: Modulo division (Case study for $m = 2^p$)

We get the last p bits of k

$m = 2^p$, $p = 1$

- $m = 2$
- $k = 567$ (Binary representation: 1000110111)
- $h(k) = 1$ (since 567 is odd)

- $m = 2$
- $k = 1234$ (Binary representation: 10011010010)
- $h(k) = 0$ (since 1234 is even)

Hash function: Modulo division (Case study for $m = 2^p$ contd.)

$m = 2^p$, $p = 3$

- $m = 2^3$
- $k = 567$ (Binary representation: 1000110111)
- $h(k) = 7$ (Binary representation: 111 $\Rightarrow$ the last three bits of 567)

- $m = 2^3$
- $k = 1234$ (Binary representation: 10011010010)
- $h(k) = 2$ (Binary representation: 010 $\Rightarrow$ the last three bits of 1234)

Hash function: Modulo division (Case study for $m = 2^p$ contd.)

$m = 2^p$, $p = 3$

- $m = 2^3$
- $k = 567$ (Binary representation: 1000110111)
- $h(k) = 7$ (Binary representation: 111 $\Rightarrow$ the last three bits of 567)

- $m = 2^3$
- $k = 1234$ (Binary representation: 10011010010)
- $h(k) = 2$ (Binary representation: 010 $\Rightarrow$ the last three bits of 1234)

- $h(k)$ is NOT dependent on the whole of k but only on its lower p bits
- It is better NOT to choose m as the power of 2

Hash function: Multiplication

Scheme

$$h(k) = \lfloor\ m\ (kA \bmod 1)\ \rfloor$$

- k is the key
- m is the number of slots
- A is a constant such that $0 < A < 1$
- $(kA \bmod 1)$ means the fractional part of kA (i.e. kA - $\lfloor\ kA\ \rfloor$)

operational steps

1. We multiply k by A and extract the fractional part (say f)
2. We multiply f by m and take its floor

Hash function: Multiplication (example)

Example 1

- $k = 1$, $m = 8$, $A = 0.61$
- $(kA \bmod 1) = (0.61 \bmod 1) = 0.61$
- Finally, $h(1) = \lfloor\ m\ (kA \bmod 1)\ \rfloor = \lfloor\ 8 * 0.61\ \rfloor = \lfloor\ 4.88\ \rfloor = 4$

Hash function: Multiplication (example)

Example 1

- $k = 1$, $m = 8$, $A = 0.61$
- $(kA \bmod 1) = (0.61 \bmod 1) = 0.61$
- Finally, $h(1) = \lfloor\ m\ (kA \bmod 1)\ \rfloor = \lfloor\ 8 * 0.61\ \rfloor = \lfloor\ 4.88\ \rfloor = 4$

Example 2

- $k = 3$, $m = 8$, $A = 0.61$
- $(kA \bmod 1) = (3*0.61 \bmod 1) = (1.83 \bmod 1) = 0.83$
- Finally, $h(3) = \lfloor\ m\ (kA \bmod 1)\ \rfloor = \lfloor\ 8 * 0.83\ \rfloor = \lfloor\ 6.64\ \rfloor = 6$

According to Dr. Knuth's suggestion, A is taken close to $(\sqrt{5}$ - $1)/2 = 0.6180339887...$

Hash function: Multiplication (features)

Features

- **Advantage**: Fairly uniform, choice of m is not critical (can be a power of 2)
- **Disadvantage**: Slower than the division method; perhaps a bit complex

Collision Resolution: Chaining

Characteristics

- All the elements that hash to the same slot are stored in a linked list for the slot
- The slot stores the head of the linked list for the slot

Insert operation

CHAINED-HASH-INSERT(T,x)
insert x at the head of the list at $T[h(x, key)]$

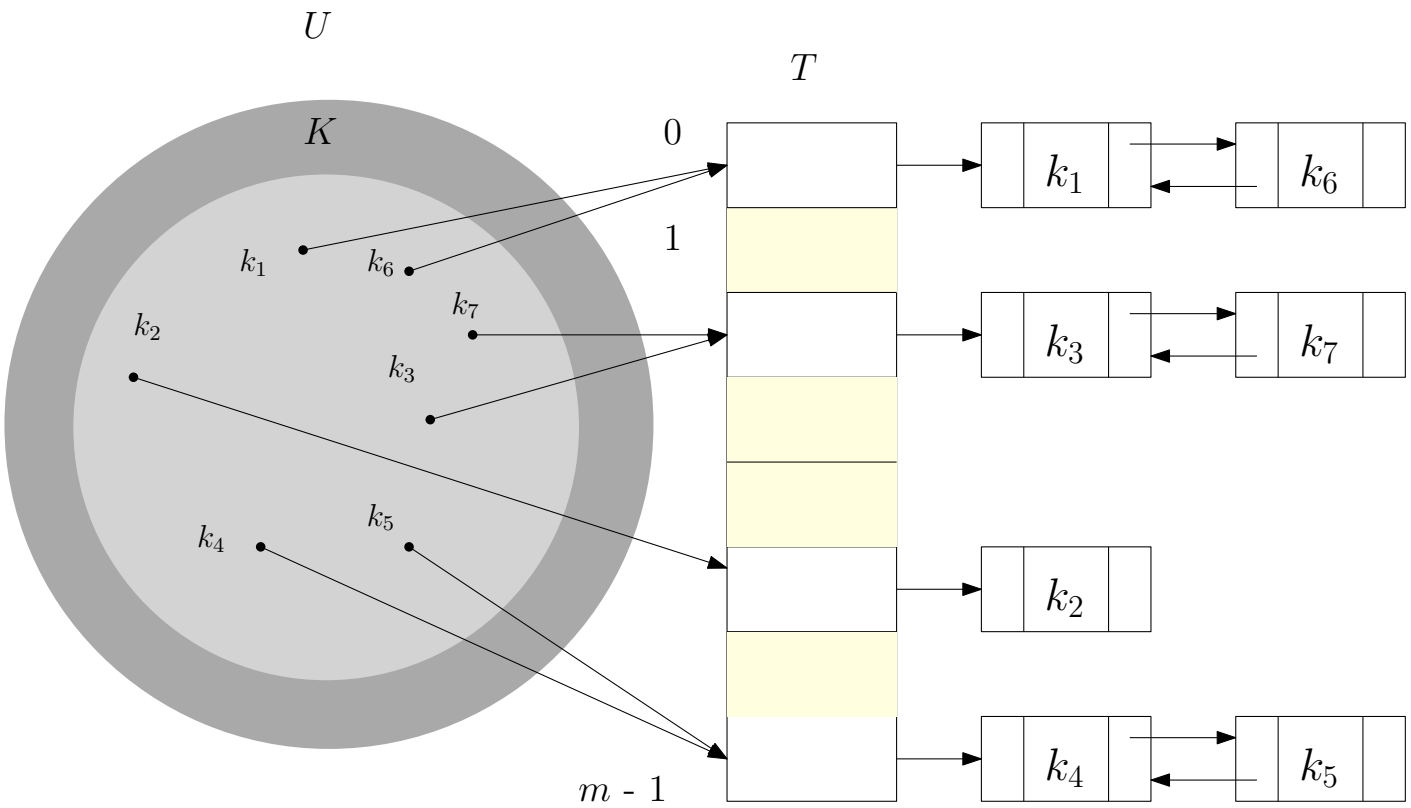
Search operation

CHAINED-HASH-SEARCH(T,k)
search for an element with key k in the list $T[h(k)]$

Delete operation

CHAINED-HASH-DELETE(T,x)
delete x from the list $T[h(x)]$

Collision Resolution: Chaining (visualization)



Collision Resolution: Chaining (example)

$h(45) = 45 \bmod 5 = 0$

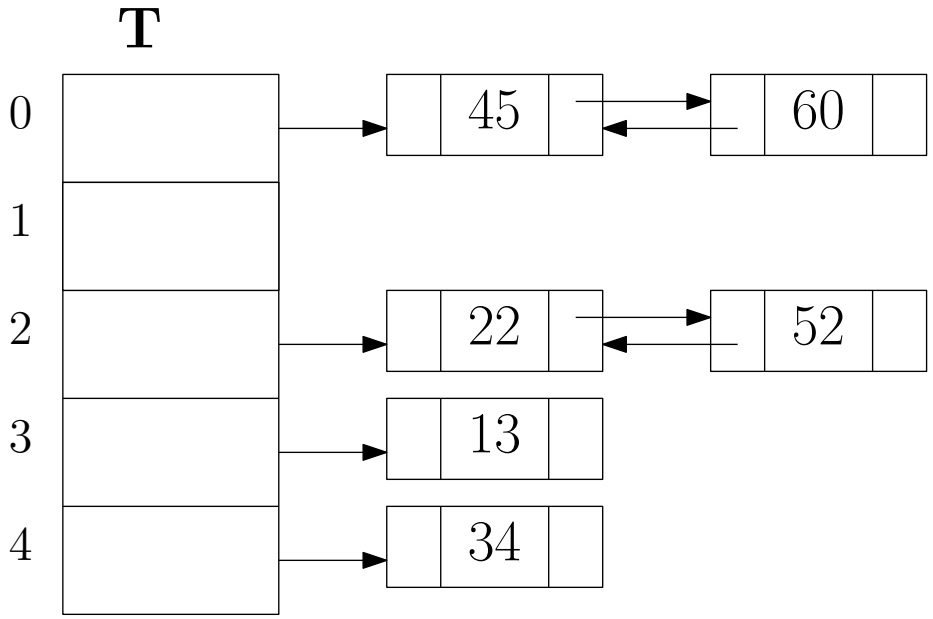
$h(34) = 34 \bmod 5 = 4$

$h(22) = 22 \bmod 5 = 2$

$h(60) = 60 \bmod 5 = 0$

$h(52) = 52 \bmod 5 = 2$

$h(13) = 13 \bmod 5 = 3$



Collision Resolution: Chaining (analysis)

Insert operation

Running time is $\mathcal{O}(1)$

Search operation

Dependent upon the size of the linked list

Delete operation

Dependent upon the size of the linked list; doubly linked lists more efficient than singly linked

Collision Resolution: Chaining (analysis contd.)

Some facts

- **Load factor** (α): Given a hash table T with m slots and n elements, *load factor* (α) is defined as n/m (the average number of elements in a chain)
- Worst case behaviour: When all the n keys are hashed to the same slot; running time $\Theta(n)$
- For uniform hashing, for successful or unsuccessful searches, where collision resolution is done by chaining, the average case complexity is $\Theta(1 + \alpha)$
  - If $n = \mathcal{O}(m)$, $\alpha = n/\mathcal{O}(m) = \mathcal{O}(1)$ (that is, searching takes constant time)

Collision Resolution: Open addressing

Characteristics

- All the elements occupy the hash table (no chaining or storage outside the table) $\Rightarrow$ load factor $(\alpha) \leq 1$
- Element inserted (or searched) in the exact slot or probed in the other slots

Probing techniques

- Linear probing
- Quadratic probing
- Double Hashing

Open addressing: Linear Probing

Scheme

$$H(k) = (h(k) + i) \bmod m$$

- k is the key
- m is the number of slots
- h is any suitable hash function (e.g. $k \bmod m$)
- H is the **auxiliary hash function**
- $i = 0, 1, 2, ..., m - 1$
- In the hash table T, we first probe $T[h(k)]$; on collision, we probe $T[h(k) + 1] \bmod m$, $T[h(k) + 2] \bmod m$ and so on
- If a slot is filled (collision occurs) the next slot is probed; mod m is to wrap round in the hash table

Linear probing (example)

✓ $h(29) = 29 \bmod 11 = 7$

address	value
0	
1	
2	
3	
4	
5	
6	
7	29
8	
9	
10	

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$

address	value
0	
1	
2	46
3	
4	
5	
6	
7	29
8	
9	
10	

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = (h(18) + 0) \bmod 11 = 7 \bmod 11 = 7$
  COLLISION!!

address	value
0	
1	
2	46
3	
4	
5	
6	
7	29
8	
9	
10	

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = (h(18) + 0) \bmod 11 = 7 \bmod 11 = 7$
  COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$

address	value
0	
1	
2	46
3	
4	
5	
6	
7	29
8	18
9	
10	

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$ COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$

address	value
0	
1	
2	46
3	36
4	
5	
6	
7	29
8	18
9	
10	

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$ COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$
- ✓ $h(43) = H(43, 0) = 43 \bmod 11 = 10$

address	value
0	
1	
2	46
3	36
4	
5	
6	
7	29
8	18
9	
10	43

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$ COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$
- ✓ $h(43) = H(43, 0) = 43 \bmod 11 = 10$
- × $h(21) = H(21, 0) = 21 \bmod 11 = 10$ COLLISION!!

address	value
0	
1	
2	46
3	36
4	
5	
6	
7	29
8	18
9	
10	43

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$
  COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$
- ✓ $h(43) = H(43, 0) = 43 \bmod 11 = 10$
- × $h(21) = H(21, 0) = 21 \bmod 11 = 10$
  COLLISION!!
  - ✓ $H(21, 1) = (h(21) + 1) \bmod 11 = (10 + 1) \bmod 11 = 0$

address	value
0	21
1	
2	46
3	36
4	
5	
6	
7	29
8	18
9	
10	43

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$ COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$
- ✓ $h(43) = H(43, 0) = 43 \bmod 11 = 10$
- × $h(21) = H(21, 0) = 21 \bmod 11 = 10$ COLLISION!!
  - ✓ $H(21, 1) = (h(21) + 1) \bmod 11 = (10 + 1) \bmod 11 = 0$
- × $h(24) = H(24, 0) = 24 \bmod 11 = 2$ COLLISION!!
  - × $H(24, 1) = (h(24) + 1) \bmod 11 = (2 + 1) \bmod 11 = 3$ COLLISION!!

address	value
0	21
1	
2	46
3	36
4	
5	
6	
7	29
8	18
9	
10	43

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$ COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$
- ✓ $h(43) = H(43, 0) = 43 \bmod 11 = 10$
- × $h(21) = H(21, 0) = 21 \bmod 11 = 10$ COLLISION!!
  - ✓ $H(21, 1) = (h(21) + 1) \bmod 11 = (10 + 1) \bmod 11 = 0$
- × $h(24) = H(24, 0) = 24 \bmod 11 = 2$ COLLISION!!
  - × $H(24, 1) = (h(24) + 1) \bmod 11 = (2 + 1) \bmod 11 = 3$ COLLISION!!
  - ✓ $H(24, 2) = (h(24) + 2) \bmod 11 = (2 + 2) \bmod 11 = 4$

address	value
0	21
1	
2	46
3	36
4	24
5	
6	
7	29
8	18
9	
10	43

Linear probing (example contd.)

- ✓ $h(29) = 29 \bmod 11 = 7$
- ✓ $h(46) = 46 \bmod 11 = 2$
- × $h(18) = H(18, 0) = 18 \bmod 11 = 7$ COLLISION!!
  - ✓ $H(18, 1) = (h(18) + 1) \bmod 11 = (7 + 1) \bmod 11 = 8$
- ✓ $h(36) = 36 \bmod 11 = 3$
- ✓ $h(43) = H(43, 0) = 43 \bmod 11 = 10$
- × $h(21) = H(21, 0) = 21 \bmod 11 = 10$ COLLISION!!
  - ✓ $H(21, 1) = (h(21) + 1) \bmod 11 = (10 + 1) \bmod 11 = 0$
- × $h(24) = H(24, 0) = 24 \bmod 11 = 2$ COLLISION!!
  - × $H(24, 1) = (h(24) + 1) \bmod 11 = (2 + 1) \bmod 11 = 3$ COLLISION!!
  - ✓ $H(24, 2) = (h(24) + 2) \bmod 11 = (2 + 2) \bmod 11 = 4$
- ✓ $h(23) = 23 \bmod 11 = 1$

address	value
0	21
1	23
2	46
3	36
4	24
5	
6	
7	29
8	18
9	
10	43

Linear probing (example contd.)

(contd.)

- × $h(54) = H(54, 0) = 54 \bmod 11 = 10$ COLLISION!!
  - × $H(54, 1) = (h(54) + 1) \bmod 11 = (10 + 1) \bmod 11 = 0$ COLLISION!!
  - × $H(54, 2) = (h(54) + 2) \bmod 11 = (10 + 2) \bmod 11 = 1$ COLLISION!!
  - × $H(54, 3) = (h(54) + 3) \bmod 11 = (10 + 3) \bmod 11 = 2$ COLLISION!!
  - × $H(54, 4) = (h(54) + 4) \bmod 11 = (10 + 4) \bmod 11 = 3$ COLLISION!!
  - × $H(54, 5) = (h(54) + 5) \bmod 11 = (10 + 5) \bmod 11 = 4$ COLLISION!!
  - ✓ $H(54, 6) = (h(54) + 6) \bmod 11 = (10 + 6) \bmod 11 = 5$

address	value
0	21
1	23
2	46
3	36
4	24
5	54
6	
7	29
8	18
9	
10	43

Primary Clustering!!

Linear probing (features)

- **Primary clustering**:
  - When the hash table is about half-filled, groups of values are stored in consecutive slots (in our example for key 54, we get a cluster: 10, 0, 1, 2, 3, 4, 5

Linear probing (features)

- **Primary clustering**:
  - When the hash table is about half-filled, groups of values are stored in consecutive slots (in our example for key 54, we get a cluster: 10, 0, 1, 2, 3, 4, 5 $\Rightarrow$ the value is stored far away from its desired slot

Linear probing (features)

- **Primary clustering**:
  - When the hash table is about half-filled, groups of values are stored in consecutive slots (in our example for key 54, we get a cluster: 10, 0, 1, 2, 3, 4, 5 $\Rightarrow$ the value is stored far away from its desired slot $\Rightarrow$ search time increases !!
- Search operation for a key starts at the initial probe, followed by the next consecutive locations until value found or an empty slot is found or the table is exhausted (searched failed on the last two cases)
- Deletion operation is similar to the search operation; to prevent a search from terminating at a deleted (hence emptied) slot, necessary action needs to be taken (a flag?)

Open addressing: Quadratic Probing

Scheme

$$H(k) = (h(k) + i^2) \bmod m$$

- k is the key
- m is the number of slots
- h is any suitable hash function (e.g. $k \bmod m$)
- H is the **auxiliary hash function**
- $i = 0, 1, 2, ..., m - 1$
- In the hash table T, we first probe $T[h(k)]$; on collision, we probe $T[h(k) + 1] \bmod m$, $T[h(k) + 4] \bmod m$, $T[h(k) + 9] \bmod m$, $T[h(k) + 16] \bmod m$ and so on
- If a slot is filled (collision occurs) the next slot in the probe sequence is probed; mod m is to wrap round in the hash table

Quadratic probing (example)

✓ $h(46) = 46 \bmod 11 = 2$

address	value
0	
1	
2	46
3	
4	
5	
6	
7	
8	
9	
10	

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$

address	value
0	
1	
2	46
3	
4	
5	
6	28
7	
8	
9	
10	

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$

address	value
0	
1	
2	46
3	
4	
5	
6	28
7	
8	
9	
10	21

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0^2) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!

address	value
0	
1	
2	46
3	
4	
5	
6	28
7	
8	
9	
10	21

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0^2) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$

address	value
0	
1	
2	46
3	35
4	
5	
6	28
7	
8	
9	
10	21

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0^2) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$
- × $h(57) = H(57, 0) = (h(57) + 0^2) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!

address	value
0	
1	
2	46
3	35
4	
5	
6	28
7	
8	
9	
10	21

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0^2) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$
- × $h(57) = H(57, 0) = (h(57) + 0^2) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!
  - × $H(57, 1) = (h(57) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$
    COLLISION!!

address	value
0	
1	
2	46
3	35
4	
5	
6	28
7	
8	
9	
10	21

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0^2) \bmod 11 = 2 \bmod 11 = 2$ COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$
- × $h(57) = H(57, 0) = (h(57) + 0^2) \bmod 11 = 2 \bmod 11 = 2$ COLLISION!!
  - × $H(57, 1) = (h(57) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$ COLLISION!!
  - × $H(57, 2) = (h(57) + 2^2) \bmod 11 = (2 + 4) \bmod 11 = 6$ COLLISION!!

address	value
0	
1	
2	46
3	35
4	
5	
6	28
7	
8	
9	
10	21

Quadratic probing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0^2) \bmod 11 = 2 \bmod 11 = 2$ COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$
- × $h(57) = H(57, 0) = (h(57) + 0^2) \bmod 11 = 2 \bmod 11 = 2$ COLLISION!!
  - × $H(57, 1) = (h(57) + 1^2) \bmod 11 = (2 + 1) \bmod 11 = 3$ COLLISION!!
  - × $H(57, 2) = (h(57) + 2^2) \bmod 11 = (2 + 4) \bmod 11 = 6$ COLLISION!!
  - ✓ $H(57, 3) = (h(57) + 3^2) \bmod 11 = (2 + 9) \bmod 11 = 0$

address	value
0	57
1	
2	46
3	35
4	
5	
6	28
7	
8	
9	
10	21

Secondary Clustering (46, 35, 57) !!

Quadratic probing (example contd.)

contd.

- × $h(39) = H(39, 0) = (h(39) + 0^2) \bmod 11 = 6 \bmod 11 = 6$
  COLLISION!!
  - ✓ $H(39, 1) = (h(39) + 1^2) \bmod 11 = (6 + 1) \bmod 11 = 7$

address	value
0	57
1	
2	46
3	35
4	
5	
6	28
7	39
8	
9	
10	21

Quadratic probing (example contd.)

contd.

- × $h(39) = H(39, 0) = (h(39) + 0^2) \bmod 11 = 6 \bmod 11 = 6$
  COLLISION!!
  - ✓ $H(39, 1) = (h(39) + 1^2) \bmod 11 = (6 + 1) \bmod 11 = 7$
- ✓ $h(19) = 19 \bmod 11 = 8$

address	value
0	57
1	
2	46
3	35
4	
5	
6	28
7	39
8	19
9	
10	21

Quadratic probing (example contd.)

contd.

- × $h(39) = H(39, 0) = (h(39) + 0^2) \bmod 11 = 6 \bmod 11 = 6$ COLLISION!!
  - ✓ $H(39, 1) = (h(39) + 1^2) \bmod 11 = (6 + 1) \bmod 11 = 7$
- ✓ $h(19) = 19 \bmod 11 = 8$
- × $h(50) = H(50, 0) = (h(50) + 0^2) \bmod 11 = 6 \bmod 11 = 6$ COLLISION!!
  - × $H(50, 1) = (h(50) + 1^2) \bmod 11 = (6 + 1) \bmod 11 = 7$ COLLISION!!
  - × $H(50, 2) = (h(50) + 2^2) \bmod 11 = (6 + 4) \bmod 11 = 10$ COLLISION!!
  - ✓ $H(50, 3) = (h(50) + 3^2) \bmod 11 = (6 + 9) \bmod 11 = 4$

address	value
0	57
1	
2	46
3	35
4	50
5	
6	28
7	39
8	19
9	
10	21

Secondary Clustering (28, 39, 50) !!

Quadratic probing (features)

- Found to be more efficient than linear probing
- No problem of primary clustering
- **Secondary clustering**:
  - Keys with the same initial probe position will also have the same sequence of locations (in our example for keys 46, 35 and 57, we get the same probe sequence: 2, 3, 6, 0, 7, 5

Quadratic probing (features)

- Found to be more efficient than linear probing
- No problem of primary clustering
- **Secondary clustering**:
  - Keys with the same initial probe position will also have the same sequence of locations (in our example for keys 46, 35 and 57, we get the same probe sequence: 2, 3, 6, 0, 7, 5 $\Rightarrow$ can't access all the positions of the table

Quadratic probing (features)

- Found to be more efficient than linear probing
- No problem of primary clustering
- **Secondary clustering**:
  - Keys with the same initial probe position will also have the same sequence of locations (in our example for keys 46, 35 and 57, we get the same probe sequence: 2, 3, 6, 0, 7, 5 $\Rightarrow$ can't access all the positions of the table $\Rightarrow$ insert may fail even if empty slots are available !! (problem can be mitigated by taking m as a prime number)
- Search operation for a key starts at the initial probe, followed by the next locations in the *probe sequence* until value found or an empty slot is found or the table is exhausted (searched failed on the last two cases)
- Deletion operation is similar to the search operation; to prevent a search from terminating at a deleted (hence emptied) slot, necessary action needs to be taken (a flag?)

Open addressing: Double Hashing

Scheme

$$H(k) = (h(k) + i * h'(k)) \bmod m$$

- k is the key
- m is the number of slots
- h is any suitable hash function (e.g. $k \bmod m$)
- h' is another appropriate hash function
- H is the **auxiliary hash function**
- $i = 0, 1, 2, \ldots, m - 1$
- In the hash table T, we first probe $T[h(k)]$; on collision, we probe $T[h(k) + h'(k)] \bmod m$, $T[h(k) + 2 * h'(k)] \bmod m$, $T[h(k) + 3 * h'(k)] \bmod m$, $T[h(k) + 4 * h'(k)] \bmod m$ and so on
- If a slot is filled (collision occurs) the next slot in the probe sequence is probed; mod m is to wrap round in the hash table

Open addressing: Double Hashing (choice of hash functions)

Possible forms

- $h(k) = k \bmod m$
- $h'(k) = n$ - $(k \bmod n)$ $[n \neq m]$

- h' should never be zero
- m and $h'(k)$ should be relatively prime

Example

- $h(k) = k \bmod 11$
- $h'(k) = 7$ - $(k \bmod 7)$

Double Hashing (example)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$

address	value
0	
1	
2	46
3	
4	
5	
6	28
7	
8	
9	
10	21

Double Hashing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0 * (7 - (35 mod 7))) \bmod 11 = 2 \bmod 11 = 2$
  COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1 * (7 - (35 mod 7))) \bmod 11 = (2 + 7) \bmod 11 = 9$

address	value
0	
1	
2	46
3	
4	
5	
6	28
7	
8	
9	35
10	21

Double Hashing (example contd.)

- ✓ $h(46) = 46 \bmod 11 = 2$
- ✓ $h(28) = 28 \bmod 11 = 6$
- ✓ $h(21) = 21 \bmod 11 = 10$
- × $h(35) = H(35, 0) = (h(35) + 0 * (7 - (35 mod 7))) \bmod 11 = 2 \bmod 11 = 2$ COLLISION!!
  - ✓ $H(35, 1) = (h(35) + 1 * (7 - (35 mod 7))) \bmod 11 = (2 + 7) \bmod 11 = 9$
- × $h(57) = H(57, 0) = (h(57) + 0 * (7 - (57 mod 7))) \bmod 11 = 2 \bmod 11 = 2$ COLLISION!!
  - ✓ $H(57, 1) = (h(57) + 1 * (7 - (57 mod 7))) \bmod 11 = (2 + 6) \bmod 11 = 8 \bmod 11 = 8$

address	value
0	
1	
2	46
3	
4	
5	
6	28
7	
8	57
9	35
10	21

Double Hashing (example contd.)

contd.

- × $h(39) = H(39, 0) = (h(39) + 0 * (7 - (39 mod 7))) \bmod 11 = 6 \bmod 11 = 6$ COLLISION!!
  - × $H(39, 1) = (h(39) + 1 * (7 - (39 mod 7))) \bmod 11 = (6 + 3) = 9 \bmod 11 = 9$ COLLISION!!
  - ✓ $H(39, 2) = (h(39) + 2 * (7 - (39 mod 7))) \bmod 11 = (6 + 6) = 12 \bmod 11 = 1$

address	value
0	
1	39
2	46
3	
4	
5	
6	28
7	
8	57
9	35
10	21

Double Hashing (example contd.)

contd.

- × $h(19) = H(19, 0) = (h(19) + 0 * (7 - (39 mod 7))) \bmod 11 = 8 \bmod 11 = 8$ COLLISION!!
  - × $H(19, 1) = (h(19) + 1 * (7 - (19 mod 7))) \bmod 11 = (8 + 2) = 10 \bmod 11 = 10$ COLLISION!!
  - × $H(19, 2) = (h(19) + 2 * (7 - (19 mod 7))) \bmod 11 = (8 + 4) = 12 \bmod 11 = 1$ COLLISION!!
  - ✓ $H(19, 3) = (h(19) + 3 * (7 - (19 mod 7))) \bmod 11 = (8 + 6) = 14 \bmod 11 = 3$

address	value
0	
1	39
2	46
3	19
4	
5	
6	28
7	
8	57
9	35
10	21

Double Hashing (example contd.)

contd.

- × $h(50) = H(50, 0) = (h(50) + 0 * (7 - (39 mod 7))) \bmod 11 = 6 \bmod 11 = 6$ COLLISION!!
  - × $H(50, 1) = (h(50) + 1 * (7 - (50 mod 7))) \bmod 11 = (6 + 6) = 12 \bmod 11 = 1$ COLLISION!!
  - ✓ $H(50, 2) = (h(50) + 2 * (7 - (50 mod 7))) \bmod 11 = (6 + 12) = 18 \bmod 11 = 7$

address	value
0	
1	39
2	46
3	19
4	
5	
6	28
7	50
8	57
9	35
10	21

Double Hashing (features)

- Found to be more efficient than linear/quadratic probing; if m is prime or a power of 2, performance is close to uniform
- No problem of primary/secondary clustering
- **No clustering**:

Key	Probe sequence
46	2, 5, 8, 0, 3, 6, ...
35	2, 9, 5, 1, 8, 4, ...
57	2, 8, 3, 9, 4, 10, ...

  That is, keys having the same hash ($i = 0$) value (here 2) have different probe sequences
- Search operation for a key starts at the initial probe, followed by the next locations in the *probe sequence* until value found or an empty slot is found or the table is exhausted (searched failed on the last two cases)
- Deletion operation is similar to the search operation; to prevent a search from terminating at a deleted (hence emptied) slot, necessary action needs to be taken (a flag?)
- **Disadvantage**: More complex and slower than linear or quadratic probing

Open address hashing (features)

- With load factor $\alpha < 1$ and assuming uniform hashing, the expected number of probes in an unsuccessful search is at most $1/(1 - \alpha)$
  - If the hash table is half full ($\alpha = 0.5$), the average number of probes in an unsuccessful search is at most 5
  - If the hash table is 90% full ($\alpha = 0.9$), the average number of probes in an unsuccessful search is at most 10
- With load factor $\alpha < 1$ and assuming uniform hashing, the expected number of probes in a successful search is at most $\frac{1}{\alpha} \ln \frac{1}{\alpha}$
  - If the hash table is half full ($\alpha = 0.5$), the average number of probes in an unsuccessful search is at most 1.397
  - If the hash table is 90% full ($\alpha = 0.9$), the average number of probes in an unsuccessful search is at most 2.559

THANK YOU !!!